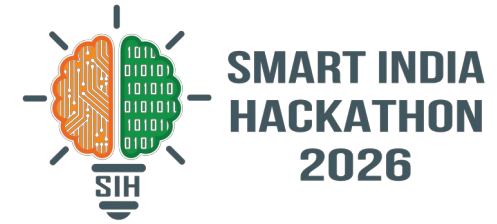


SMART INDIA HACKATHON 2026



TITLE PAGE

- **Problem Statement ID** – 26169
- **Problem Statement Title** – AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile FSOC Terminals
- **Theme** – Smart Automation
- **PS Category** – Software
- **Team ID** – —
- **Team Name** – ZeroDrift



IDEA TITLE

- ❖ *Proposed Solution (Describe your Idea/Solution/Prototype)*
- *Detailed explanation of the proposed solution*
- *How it addresses the problem*
- *Innovation and uniqueness of the solution*

The eyes and neck of a laser communication terminal

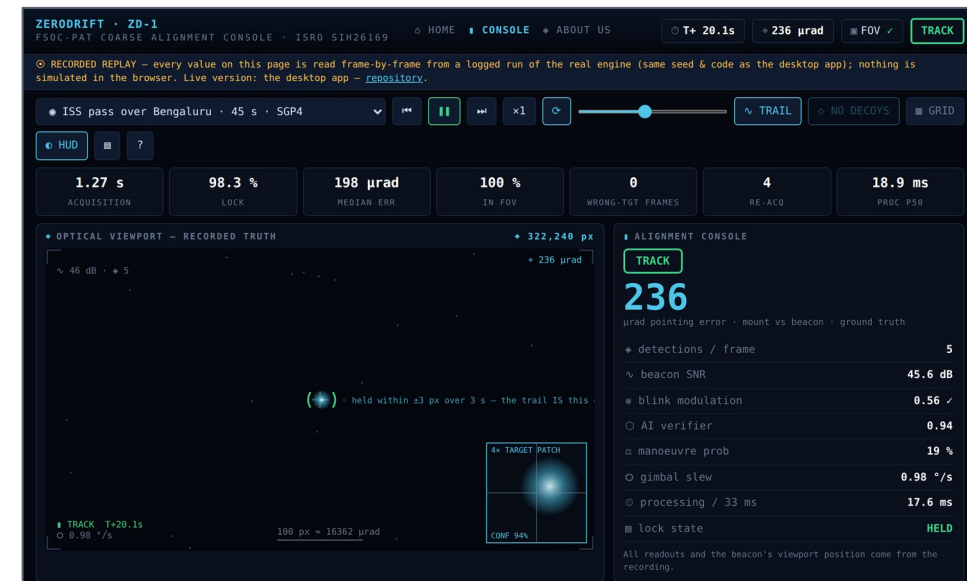
Complete virtual testbed: physics-honest sky — turbulence (Greenwood-correlated), platform vibration, sensor noise, moving beacons on real LEO/UAV paths.

AI-assisted closed-loop tracker: detects the beacon, identifies it, estimates motion, and steers a virtual pan-tilt camera to hold it in the FOV.

Addresses the problem exactly: coarse-alignment algorithms developed and validated with zero optical hardware — ISRO's stated need.

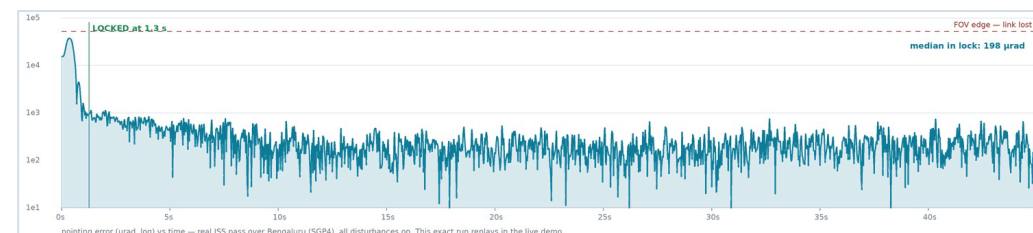
Innovation: beacon identified by its blink signature (hard gate — stars score 0), IMM motion filter, latency-compensating Smith predictor, from-scratch neural track verifier.

Not a concept — already built: GUI console, standalone app, hosted replay console, 73 automated tests, full reports — everything on this slide is measured, not projected.



Our console locked on the beacon — the same view the live demo link opens

Measured: median 198 μrad in lock · in-FOV 100% · the live-demo run



1.27 s

acquisition time

98.3 %

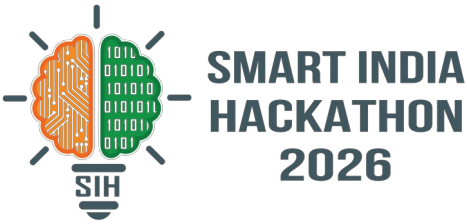
lock retention

0 / 64

wrong-target locks



TECHNICAL APPROACH



- Technologies to be used (e.g. programming languages, frameworks, hardware)
- Methodology and process for implementation (Flow Charts/Images/ working prototype)

TECH STACK

CORE

Py

NP

CV

Python 3.11 · NumPy · OpenCV

INTERFACE

Qt

PG

PySide6 console · pyqtgraph live plots

ORBITS · AI

SGP4

NN

real ISS TLEs · from-scratch neural verifier

QUALITY

73✓

CI

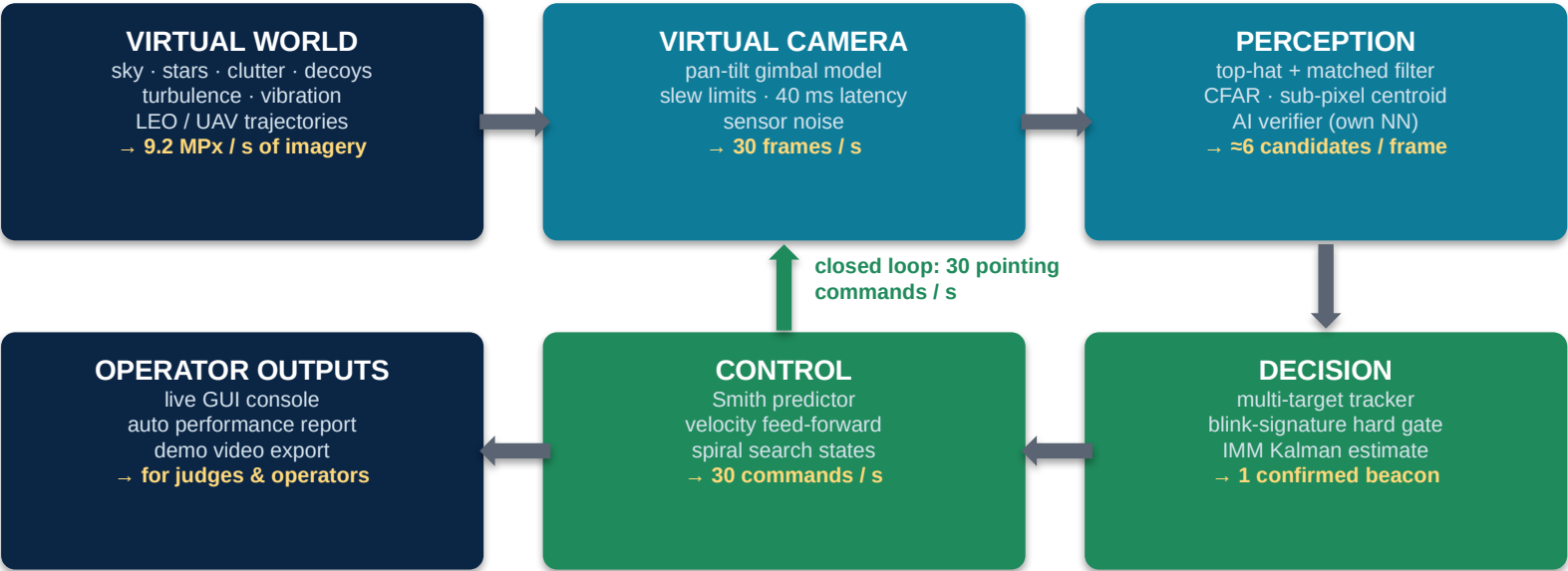
pytest suite · GitHub Actions

SHIPPING

EXE

PyInstaller one-click standalone app

100% open source · zero licensing cost



Acquisition state machine

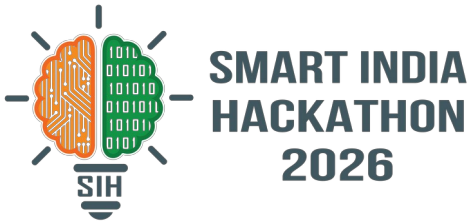


One 33 ms cycle, measured: 307,200 px in → ≈6 CFAR candidates → 1 beacon confirmed → 1 command out, ≈20 ms used (p50, ISS-pass run). Closed loop, within the 33 ms frame budget on a normal laptop; recovers even from 5° off-target among decoys.

USER FLOW AND ARCHITECTURE



FEASIBILITY AND VIABILITY



- *Analysis of the feasibility of the idea*
- *Potential challenges and risks*
- *Strategies for overcoming these challenges*

FEASIBILITY — PROVEN

- Fully working today: 73/73 automated tests pass
- 64-run randomised Monte-Carlo campaign: 100% acquisition, zero wrong locks
- Standalone Windows/Linux app builds in one click
- Tracker uses 8–21 ms of each 33 ms frame on two CPU cores — no GPU, no cloud, no dataset

CHALLENGES & RISKS

- Heavy turbulence dims and shakes the beacon
- Bright decoys / sun glints can mimic the beacon
- Mount latency destabilises fast tracking loops
- Sharp UAV manoeuvres break simple trackers

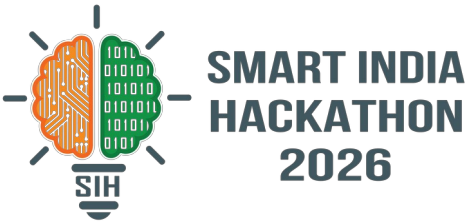
OUR STRATEGIES

- Coast-through-fade + automatic re-acquisition spiral
- Blink-signature hard gate: a star scores zero — fooled 0 times in 64 runs
- Smith predictor cancels the measured 40 ms delay
- Dual-model IMM filter — two motion models re-weighted every frame catch sharp manoeuvres

Worst case across all 64 randomised runs: slowest acquisition 4.0 s, lowest lock retention 91.6%; even a 0.53×-dim beacon under 1.74× turbulence acquired in 2.8 s. Every number regenerates from one command — fully reproducible.



IMPACT AND BENEFITS



- Potential impact on the target audience
- Benefits of the solution (social, economic, environmental, etc.)

STRATEGIC

Advances India's next-generation space & defence communications — FSOC pointing algorithms developed sovereignly, without importing optical test benches.

ECONOMIC

A laptop replaces lakhs of rupees of cameras, gimbals and optics — the entire develop–test–validate cycle for FSOC pointing runs in pure software, at zero recurring cost.

RESEARCH & EDUCATION

Any student or ISRO lab can develop, test and compare PAT algorithms instantly — reproducible scenarios, honest ground-truth scoring, open source.

SCALABLE & GREEN

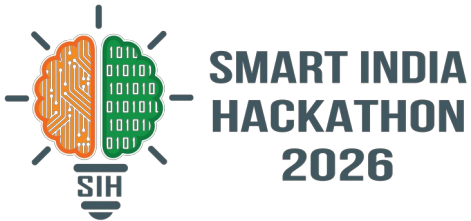
Same engine covers satellite, UAV, ground and maritime links. Pure software — no consumables, no lab energy, no e-waste in the development cycle.

Target audience: ISRO / IN-SPACe optical-communication teams, DRDO, IITs & engineering colleges, and India's growing space-tech startups.

The number that matters: on a tracked ISS pass, coarse stage + modelled fine stage closes the optical link **99.3%** of the time vs **14.7%** without — measured, not asserted.



RESEARCH AND REFERENCES



- *Details / Links of the reference and research work*

Research foundations

Kaymak et al., “A Survey on Acquisition, Tracking and Pointing Mechanisms for Mobile FSO,” IEEE Comm. Surveys & Tutorials, 2018. [\[link\]](#)

Kaushal & Kaddoum, “Optical Communication in Space: Challenges and Mitigation Techniques,” IEEE Comm. Surveys & Tutorials, 2017. [\[link\]](#)

Blackman & Popoli, Design and Analysis of Modern Tracking Systems — IMM filtering, CFAR detection, track management. [\[link\]](#)

Bar-Shalom et al., Estimation with Applications to Tracking and Navigation — Kalman/IMM theory. [\[link\]](#)

Smith, O.J.M., “Closer Control of Loops with Dead Time” — the Smith predictor, 1957. [\[link\]](#)

Vallado & Crawford, “SGP4 Orbit Determination” — AIAA 2008; real ISS TLE propagation. [\[link\]](#)

Andrews & Phillips, Laser Beam Propagation through Random Media — turbulence & scintillation models. [\[link\]](#)

Our work — open and verifiable

Repository: github.com/pranshukesavmishra/SIH-2026

Live demo (replay console): zerodrift-fsoc-pat.netlify.app

Inside: full source (5,000+ lines, documented) · 73 automated tests · 6 validated scenarios · Monte-Carlo campaign logs · technical report · user manual · demo video.

Reproducibility: every figure in this deck regenerates from one command; every random draw is logged and replayable.

Compared with existing approaches

	ZeroDrift	MATLAB sim	HW testbed
Zero hardware cost	✓	✓	✗
Real-time closed loop + live console	✓	✗	✓
Decoy & blink-signature discrimination	✓	✗	✗
Open source · one-command reproducible	✓	✗	✗

Why ZeroDrift wins on this problem

- ✓ All six mandatory deliverables already exist today
- ✓ Judged numbers, honestly measured against hidden ground truth
- ✓ AI where it earns its place — benchmarked against classical methods